Summary: Design mode

This Technical Design Report (TDR) presents the ongoing development and comparative analysis of three gas turbine engine configurations—turbojet, turbofan, and turboprop—operating in design mode. The work builds upon a previously completed and validated analysis of the turbojet engine from the prior term, which has now been finalized with confidence in the modelling approach, results, and interpretation.

The current focus expands this framework to incorporate turbofan and turboprop engines. For the turbofan, a foundational MATLAB-based design mode model has been implemented. However, further refinement is required, particularly in accurately selecting and constraining parameters such as the bypass ratio (BPR) and fan pressure ratio (FPR) to better reflect realistic engine performance. The turbofan section presents a functioning model but remains in an intermediate stage of development.

The turboprop engine analysis is at a preliminary stage. It introduces the fundamental thermodynamic process flow, engine station numbering, schematic layout, and basic performance modelling using a free turbine and propeller system. While early MATLAB implementation allows for thrust and fuel consumption estimation, detailed validation and numerical calibration—especially in relation to propeller selection criteria and propeller efficiency—are planned for future iterations. The current turboprop section therefore serves primarily as an introduction and conceptual groundwork for deeper analysis to follow.

Across all three engine types, the MATLAB code supports modular switching based on user input, reading parameter files specific to each mode. The report outlines the theoretical background, modelling assumptions, code implementation, and presents results with varying levels of completeness. The goal is to ultimately develop a unified platform capable of comparing the thermodynamic performance of all three gas turbine configurations under consistent design-mode conditions.

Table of Contents

List of Figures…...……2

List of Tables…….2

Nomenclature……...…….3

1.0 Introduction…….4

2.0 Theory and Equations…….…….4

2.1 Thermodynamic States and Processes……..…….4

1. Intake ……..…….5

2. Compressor……..……....5

3. Combustor….……..…….5

4. Turbine….……..……..…….6

5. Nozzle.……..……..……..…….6

2.2 Performance Metrics…….7

3.0 MATLAB...…….7

4.0 Results…….8

5.0 Discussion...……...10

6.0 Conclusion and Future Work.……..……...10

7.0 References….…….11

8.0 Appendices…….11

List of Figures

Figure 1: Turbojet Station Numbering……..…….4

Figure 2: Turbofan Station Numbering…….5

Figure 3: Turboprop Station Numbering……..…….5

List of Tables

Table 1: Input Parameters (Turbojet)…….……10

Table 2: Input Parameters (Turbofan)…….…….....10

Nomenclature

Symbol	Description	Units
TIT	Turbine Inlet Temperature	K
P_{o_x}	Stagnation Pressure at state point x	kPa
T_{o_x}	Stagnation Temperature at state point x	K
P_x	Static Pressure at state point x	kPa
T_x	Static Temperature at State Point x	K
V_x	Velocity at state point x	m/s
A_x	Area at state point x	m^2
ρ_x	Density at state point x	kg/m^3
h_x	Specific Enthalpy at state point x	J/kg
Q	Rate of Heat Transfer	J/s
W	Rate of Work Transfer	W

$\dot{m}$	Mass Flow Rate	kg/s
ηx	Component Isentropic Efficiency	--
LHV	Lower Heating Value	J/kg
CPR	Compressor Pressure Ratio	--

1.0 Introduction

Gas turbine propulsion systems play a vital role in the performance of modern aircraft, particularly in terms of thrust generation, fuel efficiency, and application-specific adaptability. This report builds upon prior work focused on the design mode analysis of a turbojet engine, which has been fully developed and validated using a MATLAB-based modelling framework. That analysis, centred around the results of reverse engineering results of the Jet-Cat P500 micro gas turbine (refer to T-20250403-R1-MT-TDR-CUBGT), provided detailed insight into the engine's thermodynamic behaviour at static sea-level conditions. Key performance metrics such as specific thrust, fuel-to-air ratio, and thrust-specific fuel consumption (TSFC) were extracted from the state point analysis. The turbojet's results now serve as a benchmark for further design studies.

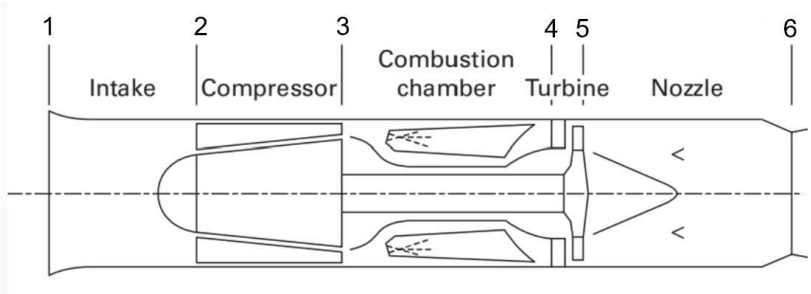

Fig 1: Turbojet station numbering

In an effort to enhance the fuel efficiency of the engine system, the next step in this study is the development of a **turbofan configuration**. A turbofan engine consists of a gas generator core—comprising a compressor, combustor, and turbine—surrounded by a bypass duct where a large fan accelerates cold, unburned air. The fan is driven by an additional low-pressure turbine stage positioned downstream of the core turbine. Compared to turbojets, turbofans typically offer **lower specific fuel consumption** (SFC) by generating a significant portion of thrust through the bypass stream. The primary motivation for the turbofan design mode model is to investigate how integrating a bypass flow—while keeping the Jet-Cat P500 as the core reference—can potentially reduce SFC by 25 % or more. While the current turbofan model includes all major components

and operating logic, further refinement is needed to accurately define parameters such as bypass ratio (BPR) and fan pressure ratio (FPR).

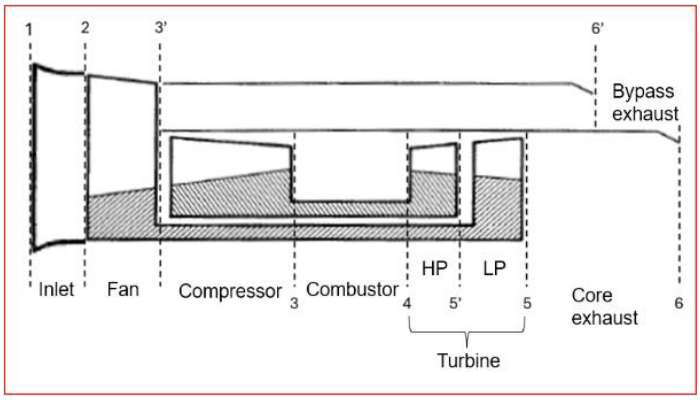

Fig 2: Turbofan Station Numbering

In addition to the turbojet and turbofan, this report introduces a **preliminary design mode model of a turboprop engine**, which offers promising performance benefits—particularly in terms of **SFC** and **propulsive efficiency at lower flight speeds**. Unlike turbojets and turbofans, the turboprop includes a **free power turbine** located downstream of the gas generator core. This turbine is **mechanically decoupled** from the core shaft and is connected to a **propeller via a reduction gearbox**, allowing it to operate at lower rotational speeds suitable for propeller-driven thrust. In a turboprop, the **majority of thrust is generated by the propeller**, rather than by jet exhaust, resulting in high levels of **static thrust**—a key advantage for short take-off and low-speed operations.

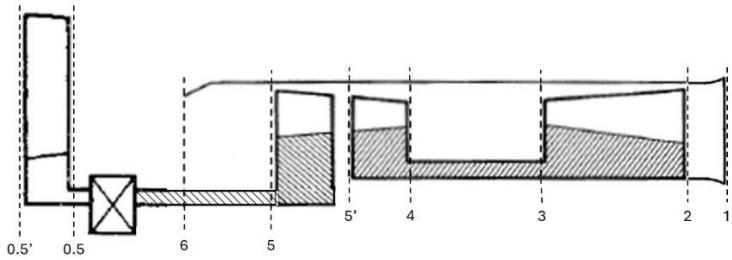

Fig 3: Turboprop Station numbering

The configuration and design of the propeller itself play a critical role in determining engine performance. Parameters such as **specific power required**, **propeller efficiency**, and **gearbox losses** significantly influence thrust and overall system behaviour. While this phase of the analysis focuses on setting up the core layout, station numbering, and basic performance equations, a more detailed thermodynamic study and performance benchmarking will be conducted in future work to fine-tune the turboprop model and assess its feasibility for low-speed aircraft applications.

2.0 Theory and Governing Equations.

This section summarizes the fundamental thermodynamic and fluid dynamic equations that govern the working of the three engine types—turbojet, turbofan, and turboprop—used in this study. For full theory and governing equations of the turbojet, refer to the previous term's TDR (T-20241204-R2-MT-TDR-CUBGT), where each component's derivations and assumptions have been provided in detail. Here, only differences relevant to the turbofan and turboprop design scripts will be highlighted.

2.1 Turbojet (Reference mode)

The turbojet model is based on conventional Brayton cycle assumptions with components modelled as idealized isentropic devices with specified efficiencies. For this study, the equations for the turbojet have been pre-established and validated. The current project builds upon those foundations to develop and contrast more complex engine configurations.

Key parameters used for turbojet:

- Isentropic efficiency for intake, compressor, turbine, nozzle
- Burner efficiency and pressure drop
- Shaft mechanical efficiency
- Performance metrics: Specific Thrust, TSFC, Thermal and Propulsive Efficiency

2.2 Turbofan Design Mode (Modifications over turbojet)

The turbofan script reuses much of the logic developed for the turbojet. However, it adds new components and modifies the station numbering. Below are the modified components and the equations governing them:

Fan:

$$P_{o_3}' = FPR * P_{o_2} \tag{1}$$

$$\text{To3}' = To_2 . \left(1 + \frac{FPR^{\frac{\gamma_{air}-1}{\gamma_{air}}} - 1}{\eta_{fan}} \right) \tag{2}$$

Compressor (Core):

$$Po_3 = \left(\frac{OPR}{FPR}\right) . Po_3' \tag{3}$$

$$To_3 = To_3'.\left(1 + \frac{\left[\left(\frac{OPR}{FPR}\right)^{\frac{\gamma_{air}-1}{\gamma_{air}}} - 1\right]}{\eta_{HPC}}\right) \quad (4)$$

LP Turbine:

$$To_5 = To_5' - \left[(BPR+1).\left(\frac{C_{p_{air}}}{\eta_m.C_{p_{gas}}}\right).(To_3' - To_2)\right) \quad (5)$$

$$Po_5 = Po_5'.\left(\frac{To_{5_s}}{T_{o_5}'}\right)^{\frac{\gamma_{gas}}{\gamma_{gas}-1}} \quad (6)$$

Cold Nozzle (Bypass):

$$T_6' = To_3' - \eta_{nc}.To_3'.(1 - \left(\frac{1}{FPR}\right)^{\frac{\gamma_{air}-1}{\gamma_{air}}} \quad (7)$$

$$v_6' = \sqrt{(2.C_{p_{air}}.(To_3' - T_6')} \quad (8)$$

$$P_6' = P_a \quad (9)$$

Performance output equations:

$$m_{total} = \frac{\left(F_{total}.(1+BPR)\right)}{\left[v_6.(1+f) + v_6'.BPR - v_{infi}.(1+BPR)\right]} \quad (10)$$

$$m_{fuel} = f.m_{hot} \quad (11)$$

$$SFC = \frac{m_{fuel}.3600}{F_{total}} \quad (12)$$

$$F_{hot} = m_{hot}.\left((1+f).v_6 - v_o\right) \quad (13)$$

$$F_{cold} = m_{cold}.(v_6' - v_o) \quad (14)$$

2.3 Turboprop (Preliminary Setup):

The turboprop design mode has not been analysed in full depth yet, but a preliminary setup has been constructed.

Key Differences from Turbojet and Turbofan:

- Includes a free power turbine (not connected to core shaft) to drive the propeller.
- Power output of free turbine is converted to shaft power using a reduction gearbox.
- Propeller produces the majority of thrust, and nozzle contribution is relatively minor.

Static thrust ($v_o = 0$) of a propeller can be estimated using:

$$F_{prop} = \eta_{prop}\sqrt{0.5.SP_{prop}^2.\rho.\pi.d_{prop}^2}\ \#14$$

Dynamic thrust ($v_o \neq 0$) of a propeller is given by:

$$F_{prop} = 4.391.10^{-8}.RPM.\frac{d_{prop}^{3.5}}{\sqrt{pitch}}.(4.233.10^{-4}.RPM.pitch - v_o)\#15$$

For the thrust, shaft power:

$$To_{5'} = \frac{SP_{turb}}{\dot{m}_{tot}.C_{p_g}} - T_{o_5}\#16$$

$$P_{shaft} = SP_{turb}.\eta_{gb}\ \#17$$

$$P_{thrust} = \eta_{prop}.P_{shaft}\ \#18$$

3.0 MATLAB

This project uses a MATLAB script to simulate the performance of three engine types: turbojet, turbofan, and turboprop. The script is simple to use. It asks the user to pick an engine type, then creates a text file where all input values can be entered. Once the inputs are filled, the script runs the engine model and gives the outputs.

The script is user-friendly and doesn't need any coding knowledge to use. It is well tested for the turbojet, which gives reliable results for thrust, fuel use, and efficiency.

The turbofan mode is built on the turbojet code. It works with a fixed thrust of 500 N and gives results for both fan and core parts. But some values like bypass ratio and fan pressure ratio still need tuning.

The turboprop model is still in early development. It includes basic functions and introduces the propeller, which is powered by a free turbine. More work will be done later to improve the accuracy.

The full MATLAB code is included in the Appendix for reference.

4.0 Results and Discussion

The turbojet results are based on the sensitivity studies TDR. These studies showed that SFC can be reduced by up to **19%** when using a compressor pressure ratio (CPR) of **8.1**, and by **11%** when using a CPR of **3.6**. These improvements reflect

how compressor efficiency and pressure ratio strongly affect overall engine performance. Refer to the T-2025-04-08-R1-HS-TDR-CUBGT(Sensitivity) for full details.

Parameter	Values
η_i	0.903
η_c	0.8315
η_b	0.9125
η_t	0.7851
η_j	0.95
η_m	0.99
f_b	0.037

Fixed	Values
CPR	3.6
$TIT\ (K)$	1148.65
V_0	17.67 m/s
F	500 N
$Altitude$	0
Mach	0

- $CPR = 3.6$

η_i	0.903
η_j	0.95
η_m	0.99
η_c	0.907
η_t	0.856
TIT	1148.65 K
f_b	0.037
η_b	0.99

$SFC = 0.134$ kg/N.hr
(11.26% reduction)

$CPR = 8.1\ (NASA\ paper)$

η_i	0.903
η_j	0.95
η_m	0.99
η_c	0.907
η_t	0.856
TIT	1148.65 K
f_b	0.037
η_b	0.99

$SFC = 0.1227$ Kg/N.hr
(18.74% reduction)

Table 1: Input parameters (left) and results for Turbojet

For the turbofan, the current results are based on the design mode script using a fixed thrust of **500 N**. With proper selection and tuning of **BPR and FPR**, the turbofan is expected to reduce SFC by **well over 25%** compared to the baseline turbojet configuration. This makes the turbofan a promising option for improving efficiency in future designs.

Parameters	Values
η_i	0.903
η_{fan}	0.85
η_c	0.8315
η_b	0.9125
η_{LPT}	0.8
η_{HPT}	0.7851
η_{n_c}	0.9
η_{n_h}	0.95

Fixed	Values
η_m	0.99
TIT (K)	1148.65
FPR	1.2
CPR	3.6
OPR	4.32
BPR	2

Table 2: Input parameters for Turbofan

Using these values, the SFC was found to be 0.1024 kg/N.hr (-32.8%).

The turboprop is still in the early stage, so no complete results are presented here. Future work will focus on refining the input values and models to get realistic thrust and SFC predictions.

5.0 Future Work

In future work, the aim to fix the nacelle area for the turbofan engine. Since the mass flow rate through the core is fixed, the total mass flow depends on the specific thrust, which is influenced by the BPR. Using the conservation of mass, the cross-sectional area of the nacelle can be calculated to maintain continuity and proper flow conditions.

$$m_{total} \approx f(BPR)$$
$$m_{core} \text{ Fixed}$$

$$Area = \frac{\dot{m}_{total}}{\rho . V}$$
$$Area = \frac{\pi . D^2}{4}$$
$$D = \sqrt{\frac{4.Area}{\pi}} * 1000\ mm$$

Fig 4: Equations for future work

6.0 References

[1] H. I. H. Saravanamuttoo, G. F. C. Rogers, H. Cohen, P. Straznicky, and A. Nix, *Gas turbine theory*, 7th ed. Harlow; New York; Toronto Pearson, 2017.

[2] M. J. Moran, H. N. Shapiro, D. D. Boettner, and M. B. Bailey, *Fundamentals of Engineering Thermodynamics, 9th Edition*. Wiley, 2018.

[3] Jet-Cat Germany, “P500-PRO-GL – Jet-Cat,” *Jetcat.de*, 2024. https://www.jetcat.de/en/productdetails/produkte/jetcat/produkte/Professionell/P500%20PRO%20-%20GL (accessed Oct. 01, 2024).

7.0 Appendix

#MATLAB Script

```
clc;
clear;

disp("Select Engine Mode:");
disp("1 - Turbojet");
disp("2 - Turbofan");
disp("3 - Turboprop");

mode = input("Enter mode number: ");

% File names based on engine mode
switch mode
    case 1
        filename = 'turbojet_params.txt';
        template = {
            'altitude = '
            'Mach = '
            'P_cr = '
            'e_i = '
            'e_c = '
            'e_t = '
            'e_m = '
            'e_n = '
            'burner_eff = '
            'pressure_drop = '
        };
    case 2
        filename = 'turbofan_params.txt';
        template = {
            'altitude = '
            'Mach = '
            'To_4 = '
            'eta_i = '
            'eta_fan = '
            'eta_HPC = '
            'eta_b = '
            'eta_LPT = '
            'eta_HPT = '
            'eta_n_c = '
            'eta_n_h = '
            'eta_m = '
            'd_pb = '
            'F_total = '
            'BPR = '
            'FPR = '
            'OPR = '
        };
    case 3
```

```
        filename = 'turboprop_params.txt';
        template = {
            'altitude = '
            'Mach = '
            'PR = '
            'TIT = '
            'SP_prop = '
            'm_air = '
        };
    otherwise
        error("Invalid mode selected. Please choose 1, 2, or 3.");
end

% If file doesn't exist, create it with placeholders
if ~isfile(filename)
    fid = fopen(filename, 'w');
    fprintf(fid, '# Please fill in the values for the following parameters\n');
    for i = 1:length(template)
        fprintf(fid, '%s\n', template{i});
    end
    fclose(fid);
    fprintf('Parameter file "%s" has been created.\n', filename);
end

% Prompt the user to fill in the values
fprintf('\nPlease edit the file "%s" with the required values.\n', filename);
fprintf('When done, save and close the file, then press any key to continue...\n');
edit(filename); % Open in MATLAB Editor
pause;          % Wait for user to hit any key

% Read parameter values
params = read_parameters(filename);

% Extract common inputs
altitude = params.altitude;
Mach = params.Mach;

% Run corresponding engine model
switch mode
    case 1
        run_turbojet_design(altitude, Mach, params);
    case 2
        run_turbofan_design(altitude, Mach, params);
    case 3
        run_turboprop_design(altitude, Mach, params);
end

function params = read_parameters(filename)
    % Initialize empty structure
    params = struct();
```

```
    % Open the file
    fid = fopen(filename, 'r');
    if fid == -1
        error('Cannot open file: %s', filename);
    end

    % Parse each line
    while ~feof(fid)
        line = strtrim(fgetl(fid));

        % Skip empty lines or comments
        if isempty(line) || startsWith(line, '#')
            continue;
        end

        % Split line into key-value
        tokens = strsplit(line, '=');
        if length(tokens) == 2
            key = strtrim(tokens{1});
            valStr = strtrim(tokens{2});
            val = str2double(valStr);
            if isnan(val)
                params.(key) = valStr;  % store as string if not numeric
            else
                params.(key) = val;
            end
        end
    end

    fclose(fid);
end

%% ================= TURBOJET DESIGN =================
function run_turbojet_design(altitude, Mach, params)
    M = Mach;
    [T_a, a, P_amb] = atmosisa(altitude);
    P_a = P_amb / 1000;
    radius = 0.143 / 2;
    A1 = pi * radius^2;
    LHV = 43260000; F = 500;

    % Parameters from input file
    P_cr = params.P_cr;
    e_i = params.e_i;
    e_c = params.e_c;
    e_t = params.e_t;
    e_m = params.e_m;
    e_n = params.e_n;
    burner_eff = params.burner_eff;
```

```
pressure_drop = params.pressure_drop;
To_4 = params.TIT;
V_inlet = params.V_inlet;

gamma_air = 1.4; gamma_gas = 1.333;
Cp_air = 1005; Cp_gas = 1148;
R = (Cp_gas*(gamma_gas-1))/gamma_gas;

T1 = T_a; P1 = P_a;
To_1 = T1 + (V_inlet^2) / (2 * Cp_air);
Po_1 = P1 * (1 + (V_inlet^2) / (2 * Cp_air * T1))^(gamma_air / (gamma_air - 1));

To_2 = To_1;
Po_2 = P1 * (1 + e_i * (V_inlet^2) / (2 * Cp_air * T1))^(gamma_air / (gamma_air - 1));

To_3 = To_2 * (1 + ((P_cr^((gamma_air - 1)/gamma_air)) - 1)/e_c);
Po_3 = P_cr * Po_2;


Po_4 = Po_3 * (1 - pressure_drop);

To_5 = To_4 - (Cp_air * (To_3 - To_2)) / (Cp_gas * e_m);
To_5s = To_4 - (To_4 - To_5) / e_t;
Po_5 = Po_4 * (To_5s / To_4)^(gamma_gas / (gamma_gas - 1));

NPR = Po_5 / P_a;
CPR = (1 - 1/e_n * ((gamma_gas - 1)/(gamma_gas + 1)))^(-gamma_gas / (gamma_gas - 1));

if NPR > CPR
    flow_status = 'Choked';
    T_6 = (2 / (gamma_gas + 1)) * To_5;
    P_6 = Po_5 / CPR;
    V6 = sqrt(gamma_gas * 287 * T_6);
    rho_6 = (P_6 *1000) / (R* T_6);
    A6_by_m = (rho_6 * V6)^(-1);
else
    flow_status = 'Unchoked';
    P_6 = P_a;
    T_6 = To_5 - (e_n * To_5 * (1 - (1 / NPR)^((gamma_gas - 1) / gamma_gas)));
    V6 = sqrt(2 * Cp_gas * (To_5 - T_6));
    rho_6 = (P_6 * 1000) / (R*T_6);
    A6_by_m = (rho_6 * V6)^(-1);
end
To_6 = T_6 + V6^2 / (2 * Cp_gas);
Po_6 = P_amb/1000;

f = (Cp_gas * To_4 - Cp_air * To_3) / (burner_eff * LHV - Cp_gas * To_4);
specific_thrust = V6 * (1 + f) - V_inlet + (P_6-P_a)*1000*A6_by_m;

m_air = F / specific_thrust;
```

```
    m_fuel = m_air * f;
    m = m_air + m_fuel;
    A6 = A6_by_m * m;
    TSFC = m_fuel * 3600 / F;

    thermal_eff = (m_air * (V6^2 - V_inlet^2) / 2) / (m_fuel * LHV);
    prop_eff = 2 * V_inlet * (V6 * (1 + f) - V_inlet) / ...
              ((1 + f) * V6^2 - V_inlet^2);
    overall_eff = thermal_eff * prop_eff;
    M1 = V_inlet/a;
    M6 = V6/a;

    %% Create state table
    states = {'1 (Inlet)', '2 (Diffuser)', '3 (Compressor)', ...
              '4 (Combustor)', '5 (Turbine)', '6 (Nozzle)'};
    static_T = [T1, NaN, NaN, NaN, NaN, T_6];
    total_T = [To_1, To_2, To_3, To_4, To_5, To_6];
    static_P = [P1, NaN, NaN, NaN, NaN, P_6];
    total_P = [Po_1, Po_2, Po_3, Po_4, Po_5, Po_6];
    velocity = [V_inlet, NaN, NaN, NaN, NaN, V6];
    Area = [A1, NaN, NaN, NaN, NaN, A6];
    Mach = [M1, NaN, NaN, NaN, NaN, M6];

    T_table = table(states', static_T', total_T', static_P', total_P', velocity', Area',
Mach', ...
        'VariableNames', {'State', 'T_static_K', 'T_total_K', 'P_static_kPa',
'P_total_kPa', 'Velocity_mps', 'Area_m^2', 'Mach no.'});

    disp('=== State Point Properties ===');
    disp(T_table);

    %% Plot total pressure and temperature profiles
    figure;
    subplot(2,1,1);
    plot(1:6, total_T, '-o','LineWidth',2);
    title('Total Temperature Profile');
    xlabel('State Point'); ylabel('Total Temperature (K)'); grid on;
    xticks(1:6); xticklabels(states);

    subplot(2,1,2);
    plot(1:6, total_P, '-o','LineWidth',2);
    title('Total Pressure Profile');
    xlabel('State Point'); ylabel('Total Pressure (kPa)'); grid on;
    xticks(1:6); xticklabels(states);

    %% Final output
    fprintf('\n=== TURBOJET PERFORMANCE ===\n');
    fprintf('Exit Velocity: %.2f m/s\n', V6);
    fprintf('Specific Thrust: %.2f Ns/kg\n', specific_thrust);
    fprintf('TSFC: %.4f kg/Nh\n', TSFC);
```

```
    fprintf('Fuel-to-Air Ratio (f): %.4f\n', f);
    fprintf('Air Mass Flow Rate: %.2f kg/s\n', m_air);
    fprintf('Fuel Mass Flow Rate: %.4f kg/s\n', m_fuel);
    fprintf('Thermal Efficiency: %.4f\n', thermal_eff);
    fprintf('Propulsive Efficiency: %.4f\n', prop_eff);
    fprintf('Overall Efficiency: %.4f\n', overall_eff);
    fprintf('Flow Type at Nozzle: %s\n', flow_status);
end


%% ================= TURBOFAN DESIGN =================
function run_turbofan_design(altitude, Mach, params)
    M = Mach;
    [T, a, P, rho] = atmosisa(altitude);
    P_a = P / 1000; T_a = T;

    % Parameters from input file
    BPR = params.BPR;
    OPR = params.OPR;
    FPR = params.FPR;
    eta_i = params.eta_i;
    eta_fan = params.eta_fan;
    eta_HPC = params.eta_HPC;
    eta_LPT = params.eta_LPT;
    eta_HPT = params.eta_HPT;
    eta_m = params.eta_m;
    eta_n_c = params.eta_n_c;
    eta_n_h = params.eta_n_h;
    eta_b = params.eta_b;
    d_pb = params.d_pb;
    To_4 = params.TIT;
    V_inf = params.V_inf;

    V1 = V_inf;

    F_total = 500;

    Cp_air = 1005; Gamma_air = 1.4;
    Cp_gas = 1148; Gamma_gas = 1.333;
    LHV = 43260000;

    T_1 = T_a; P_1 = P_a;
    To_2 = T_1 + V_inf^2 / (2 * Cp_air);
    Po_2 = P_1 * (1 + eta_i * V_inf^2 / (2 * Cp_air * T_a))^(Gamma_air / (Gamma_air - 1));

    Po_3p = FPR * Po_2;
    To_3p = To_2 * (1 + (FPR^((Gamma_air - 1) / Gamma_air) - 1) / eta_fan);

    Po_3 = (OPR / FPR) * Po_3p;
    To_3 = To_3p * (1 + ((OPR / FPR)^((Gamma_air - 1) / Gamma_air) - 1) / eta_HPC);
```

```
Po_4 = Po_3 * (1 - d_pb);
To_5p = To_4 - (Cp_air * (To_3 - To_3p)) / (Cp_gas * eta_m);
To_5sp = To_4 - (To_4 - To_5p) / eta_HPT;
Po_5p = Po_4 * (To_5sp / To_4)^(Gamma_gas / (Gamma_gas - 1));

To_5 = To_5p - (BPR + 1) * (Cp_air / (eta_m * Cp_gas)) * (To_3p - To_2);
To_5s = To_5p - ((To_5p - To_5) / eta_LPT);
Po_5 = Po_5p * (To_5s / To_5p)^(Gamma_gas / (Gamma_gas - 1));

f = (Cp_gas * To_4 - Cp_air * To_3) / (eta_b * LHV - Cp_gas * To_4);

T6p = To_3p - eta_n_c * To_3p * (1 - (1 / FPR)^((Gamma_air - 1) / Gamma_air));
V6p = sqrt(2 * Cp_air * (To_3p - T6p));
P6p = P_a;

NPR_hot = Po_5 / P_a;
T6 = To_5 - eta_n_h * To_5 * (1 - (1 / NPR_hot)^((Gamma_gas - 1) / Gamma_gas));
V6 = sqrt(2 * Cp_gas * (To_5 - T6));
P6 = P_a;

To_6 = T6 + V6^2 / (2 * Cp_gas);
Po_6 = P_a;

m_total = F_total * (1 + BPR) / (V6 * (1 + f) + V6p * BPR - V_inf * (1 + BPR));
m_hot = m_total / (1 + BPR);
m_cold = m_total - m_hot;
m_fuel = m_hot * f;

F_cold = m_cold * (V6p - V_inf);
F_hot = m_hot * ((1 + f) * V6 - V_inf);

specific_thrust = F_total / m_total;
SFC = m_fuel * 3600 / F_total;

 %% Create state table
states = {'1 (Inlet)', '2 (Diffuser)', '3 (Compressor)', '3p (by-pass)', ...
          '4 (Combustor)', '5 (Turbine)', '5p (by-pass)', '6 (Nozzle)', '6p'};
static_T = [T_1, NaN, NaN, NaN, NaN,NaN, NaN, T6, T6p];
total_T = [To_2, To_2, To_3,To_3p, To_4, To_5, To_5p To_6, NaN];
static_P = [P_1, NaN, NaN,NaN, NaN, NaN,NaN, P6, P6p];
total_P = [Po_2, Po_2, Po_3,Po_3p, Po_4, Po_5,Po_5p Po_6, NaN];
velocity = [V1, NaN, NaN, NaN,NaN, NaN, NaN V6, V6p];


T_table = table(states', static_T', total_T', static_P', total_P', velocity', ...
    'VariableNames', {'State', 'T_static_K', 'T_total_K', 'P_static_kPa',
'P_total_kPa', 'Velocity_mps',});

disp('=== State Point Properties ===');
```

```
    disp(T_table);

    %% Plot total pressure and temperature profiles
    figure;
    subplot(2,1,1);
    plot(1:9, total_T, '-o','LineWidth',2);
    title('Total Temperature Profile');
    xlabel('State Point'); ylabel('Total Temperature (K)'); grid on;
    xticks(1:9); xticklabels(states);

    subplot(2,1,2);
    plot(1:9, total_P, '-o','LineWidth',2);
    title('Total Pressure Profile');
    xlabel('State Point'); ylabel('Total Pressure (kPa)'); grid on;
    xticks(1:9); xticklabels(states);


    fprintf('\n=== TURBOFAN RESULTS ===\n');
    fprintf('Specific Thrust: %.2f N/(kg/s)\n', specific_thrust);
    fprintf('SFC: %.4f kg/hr/N\n', SFC);
    fprintf('Thrust Check: %.2f N\n', F_cold + F_hot);
end

function run_turboprop_design(altitude, Mach, params)
    M = Mach;
    [Ta, a, Pa_pa, rhoa] = atmosisa(altitude);
    Pa = Pa_pa * 1e-3;

    % Parameters from input file
    PR = params.PR;
    TIT = params.TIT;
    SP_prop = params.SP_prop;
    eta_prop = params.eta_prop;

    m_air = params.m_air;
    TP_prop = SP_prop / eta_prop;

    [To2, Po2, P1, T1, v1] = turboprop_intake(Ta, Pa, m_air, rhoa, params);
    [To3, Po3] = turboprop_compressor(PR, To2, Po2, params);
    [To4, Po4] = turboprop_burner(Po3, TIT, params);
    [To5p, Po5p] = turboprop_turbine(To4, To3, To2, Po4, params);
    [To5, Po5, SP_turb, f, m_tot] = turboprop_freeturbine(SP_prop, m_air, To5p, To4, Po5p,
To3, params);
    [F_core, SFC, T6, v6] = turboprop_performance(v1, Pa, f, To5, Po5, m_air, params);

    fprintf('\n=== TURBOPROP RESULTS ===\n');
    fprintf('Thrust: %.2f N\n', F_core);
    fprintf('SFC: %.4f kg/Nh\n', SFC);
    fprintf('Exit Velocity: %.2f m/s\n', v6);
end
```

```
%% ========== TURBOPROP FUNCTIONS ==========

function [To2, Po2, P1, T1, v1] = turboprop_intake(Ta, Pa, m_air, rhoa, params)
    eta_i = params.eta_i; ka = 1.4; Cpa = 1005;
    A1 = pi * (74.355 / 2000)^2;
    v1 = m_air / (rhoa * A1);
    P1 = Pa;
    T1 = Ta;
    To1 = T1 + v1^2 / (2 * Cpa);
    To2 = To1;
    Po2 = P1 * (1 + eta_i * (v1^2 / (2 * Cpa * T1)))^(ka / (ka - 1));
end

function [To3, Po3] = turboprop_compressor(PR, To2, Po2, params)
    eta_c = params.eta_c; ka = 1.4;
    Po3 = PR * Po2;
    To3 = To2 * (1 + (PR^((ka - 1) / ka) - 1) / eta_c);
end

function [To4, Po4] = turboprop_burner(Po3, TIT, params)
    fb = params.fb;
    Po4 = Po3 * (1 - fb);
    To4 = TIT;
end

function [To5p, Po5p] = turboprop_turbine(To4, To3, To2, Po4, params)
    eta_t = params.eta_t; eta_m = params.eta_m; Cpa = 1005;
    kg = 1.333; Cpg = 1148;
    To5p = To4 - (Cpa * (To3 - To2)) / (Cpg * eta_m);
    To5ps = To4 - (To4 - To5p) / eta_t;
    Po5p = Po4 * (To5ps / To4)^(kg / (kg - 1));
end

function [To5, Po5, SP_turb, f, m_tot] = turboprop_freeturbine(SP_prop, m_air, To5p, To4,
Po5p, To3, params)
    Cpa = 1005; Cpg = 1148; kg = 1.333;
    LHV = 43.26e6; eta_t = 0.783902; eta_b = params.eta_b; eta_gb = params.eta_gb;
    f = (Cpg * To4 - Cpa * To3) / (eta_b * LHV - Cpg * To4);
    m_tot = m_air * (1 + f);
    SP_turb = SP_prop / eta_gb;
    To5 = To5p - SP_turb / (m_tot * Cpg);
    To5s = To5p - ((To5p - To5) / eta_t);
    Po5 = Po5p * (To5s / To5p)^(kg / (kg - 1));
end

function [F_core, SFC, T6, v6] = turboprop_performance(v1, Pa, f, To5, Po5, m_air, params)
    kg = 1.333; LHV = 43.26e6; Cpg = 1148;
    Rg = (Cpg * (kg - 1)) / kg;
    eta_n = params.eta_n;
```

```
    PRn = Po5 / Pa;
    CrPR = 1 / (1 - (1 / eta_n) * ((kg - 1) / (kg + 1)))^(kg / (kg - 1));

    if PRn > CrPR
        T6 = To5 * (2 / (kg + 1));
        P6 = Po5 / CrPR;
        rho6 = (1000 * P6) / (Rg * T6);
        v6 = sqrt(kg * Rg * T6);
        sA6 = 1 / (rho6 * v6);
        Fs = ((1 + f) * v6 - v1) + sA6 * (P6 - Pa) * 1e3;
    else
        P6 = Pa;
        T6 = To5 * (1 - eta_n * (1 - (P6 / Po5)^((kg - 1) / kg)));
        v6 = sqrt(2 * Cpg * (To5 - T6));
        rho6 = (1000 * P6) / (Rg * T6);
        sA6 = 1 / (rho6 * v6);
        Fs = ((1 + f) * v6 - v1) + sA6 * (P6 - Pa) * 1e3;
    end

    F_core = m_air * Fs;
    SFC = (f * 3600) / Fs;
end
```